

環境音シーン認識・安全アシストデバイス

聞こえない危険を、光と振動で知らせるエッジAIウェアラブル

micro:bit v2 + μ T-Kernel 3.0 + 小型CNN

渡邊 和弥（ワタナベ カズヤ）

京都工芸繊維大学 工芸科学部 電子システム工学課程 3年

気づけない「危険を知らせる音」

サイレンやクラクションは、 耳で受け取る「警告」です

- 難聴・聴覚障害のある人は、背後から近づく緊急車両に気づきにくい
- ノイズキャンセリングヘッドホンを使っていると、周囲の警告音が届かない
- 音の警告は「その場で」気づけなければ意味がない

緊急車両のサイレン

道をあける・止まるなどの判断が必要

クラクション

とっさによける判断が必要

声やドアの音も同じくらい大きいため、
音量ではなく「音の種類」を
聞き分ける必要がある。

そこでAIを使う。

聞いて、判別して、知らせる

1. 聞く

micro:bitの内蔵マイクで、
周りの音を1秒に8,000回の
速さで取り込み続ける

2. 判別する

小型のAI (CNN) が1秒ごとに
「サイレン／クラクション／
その他」を判定する

3. 知らせる

危険な音のときだけ、
LEDの点灯と振動で知らせる

- クラウドを使わず、音声を端末の外に出さない
- 電池で動き、腕に着けて持ち歩ける
- 音の大きさではなく、音の種類で判別する

動作のようす



デモ動画 : <https://youtu.be/pyXOdZmFe1g>

	検知した音	LED	振動
●	サイレン	一番上の段	長く2回
●	クラクション	真ん中の段	短く3回
●	その他・静音	点灯しない	なし

シリアル出力（実機のログ）

```
=> other conf=652 (340ms) run=3
=> SIREN conf=882 (340ms) run=2 *** ALERT ***
=> SIREN conf=908 (340ms) run=7 *** ALERT ***
```

ハードウェア構成

micro:bit v2

プロセッサ nRF52833

Cortex-M4F 64MHz | RAM 128KB | Flash 512KB

内蔵MEMSマイク（アナログ出力）

SAADC + EasyDMAで8kHz取り込み（ドライバを自作）

5×5 LEDマトリクス

危険音の種類を点灯位置で表示

P0で制御

振動モーターモジュール

ドライバ（トランジスタ）内蔵。P0・3V・GNDの端子にM3ネジで固定

電源

電池ボックス（単4×2本）

約3Vで動作。スイッチ付き

外付けの部品は振動モーターと電池だけで、音の取り込みから判別・通知までをmicro:bit 1枚で行う。

録音を止めずに推論する（μT-Kernelのタスク設計）

T_audio 録音

優先度 2（高）

SAADCをピンポンバッファで連続駆動。
1秒分たまるたびに推論タスクへ知らせる

↓ イベントフラグ（tk_set_flg）

T_infer 推論

優先度 4（低）

特徴量の計算とCNN推論。約350msかかる、いちばん重い処理

↓ メッセージバッファ（tk_snd_mbf）

T_notify 通知

優先度 3

通知の条件を満たした危険音だけ、LEDと振動で知らせる

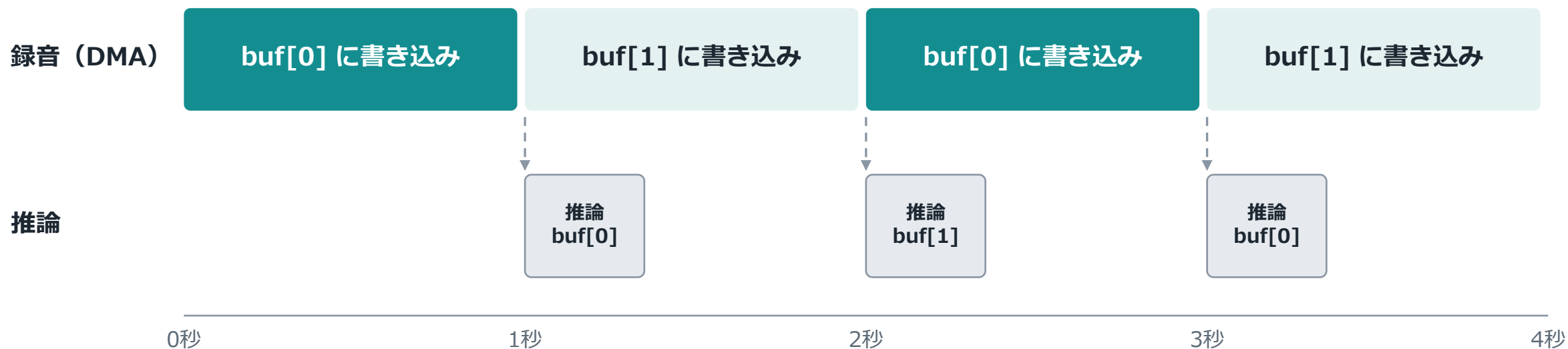
設計の意図

- 推論中に鳴った音も取りこぼさない
- いちばん重い推論を、いちばん低い優先度に
- 録音はいつでも推論に割り込める

約35%

推論が使うCPUの割合

推論中でも録音を続ける仕組み（ピンポンバッファ）



- SAADCの書き込み先は予約できる。STARTEDで次の面を登録し、ENDの直後にSTARTして切り替える
- 推論（約0.35秒）は次の1秒がたまるより早く終わるので、読んでいる面が上書きされない
- ENDの検出は2ms周期のため、切り替え時にできる空白は最大約2ms

1秒の音を3クラスに分類するまで



モデル

- 畳み込み 8→16→32 + 平均プーリング + 全結合
- パラメータ 7,043個 (約28KB)
- 入力 960次元 (32メル帯域 × 30フレーム)

実機への載せ方

- Kerasで学習し、重みをCの配列に書き出す
- 推論ライブラリを使わず、手書きのCで実装
- PythonとCの出力が一致 (誤差 4.77e-7)

工夫① 音量が変わっても判定がぶれない

問題

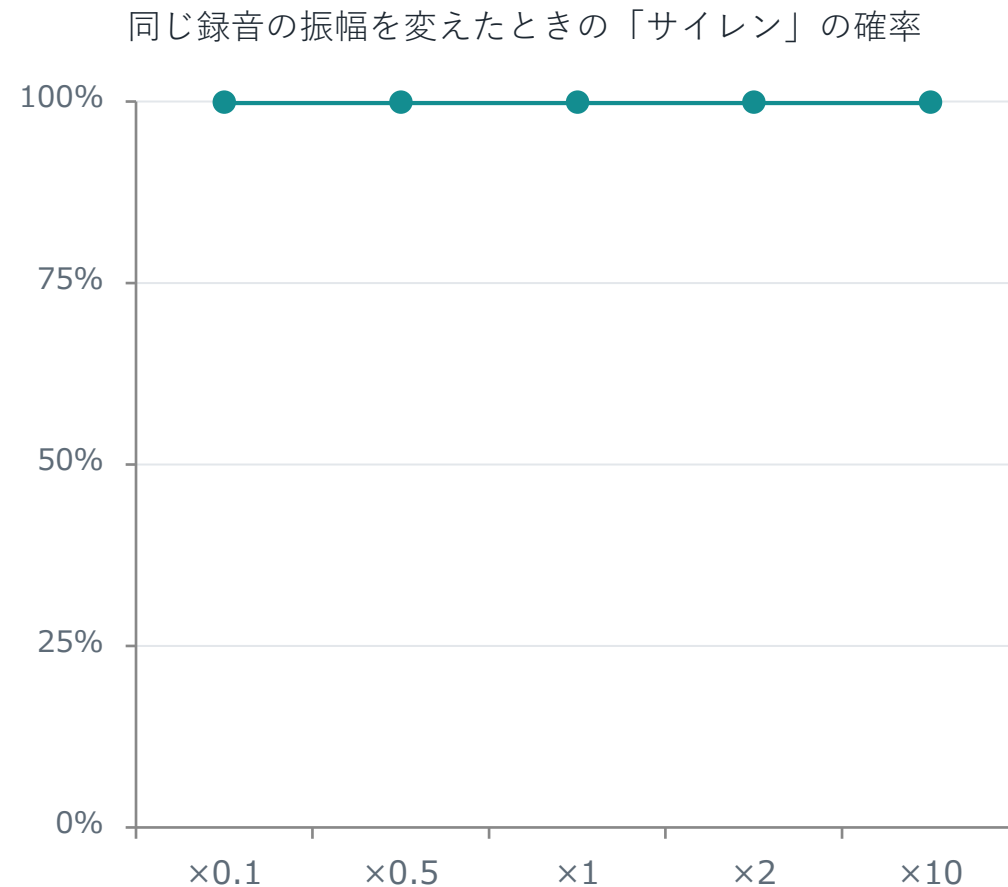
固定のゲインで取り込むと、マイクの感度や音源との距離で特徴量全体がずれ、実機でサイレンをクラクションと誤判定していた

対策

1秒ごとにDC成分を取り除き、RMS（音の強さ）を基準値にそろえてから特徴量を作る。
学習側と実機側の両方に同じ処理を入れた

結果

同じ録音の振幅を0.1倍～10倍に変えても、出力確率はほぼ同じ（差は最大 $5.2e-10$ ）。
マイクゲインの手動較正も不要に



工夫② 実機のマイクで録った音でも学習

公開データだけでは、日本のサイレンの音程やmicro:bitのマイクの特性に合わない

公開データセット

ESC-50・UrbanSound8K
(サイレン929、クラクション429、その他400クリップ)

日本のサイレン

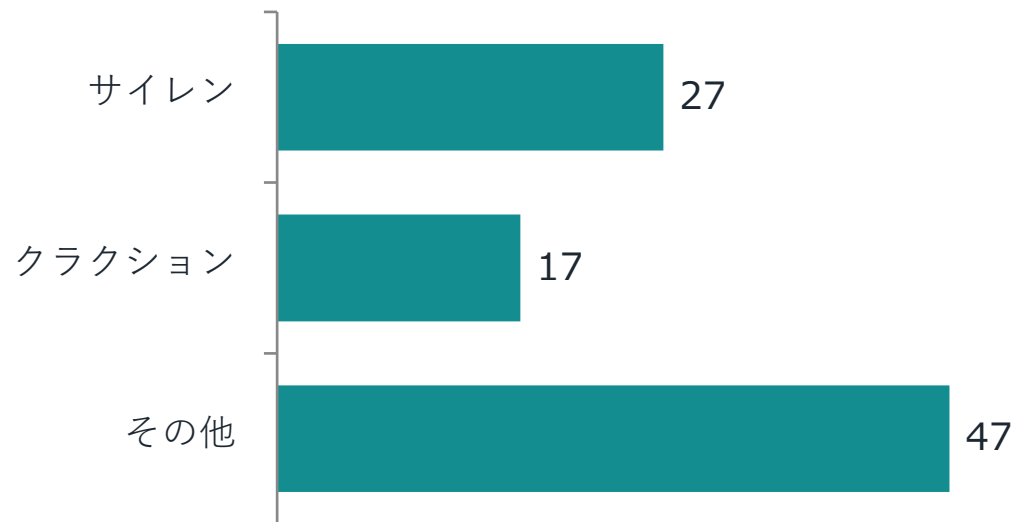
OtoLogic (CC BY 4.0) とDOVA-SYNDROMEの
救急車・消防車・パトカーの音 43本

実機で録音

micro:bitの内蔵マイクで67本・91秒を自作。
雑音を混ぜたデータを多めに作り、学習での比重を高めた

音声データは再配布が制限されているため公開せず、出典をCREDITS.mdに明記

実機録音データ (1秒窓の数)



録音用のプログラムも自作

音を検知すると自動で1秒録音してPCに保存。
無音の録音は品質チェックで除外した

評価：実機データで誤報ゼロ

90.1%

実機のマイクで録った音の正解率（91窓）

0件

危険でない音を危険と判定した件数（その他47窓）

100%

「サイレン」と判定したときの的中率

混同行列（micro:bitで録った91窓・5分割交差検証）

正解 \ 予測	サイレン	クラクション	その他
サイレン	23	2	2
クラクション	0	12	5
その他	0	0	47

- 同じ録音が学習とテストに混ざらないよう、ファイル単位で分割
- 公開データ6,001窓：クリーン87.2%、雑音入り82.5%
- 弱点：短いクラクションの取りこぼし（再現率 70.6%）

小さなメモリで、リアルタイムに動く

約350ms

1秒の音あたりの推論時間
(実測340~380ms)

約103KB

RAMのうち静的に確保する量
(128KB中)

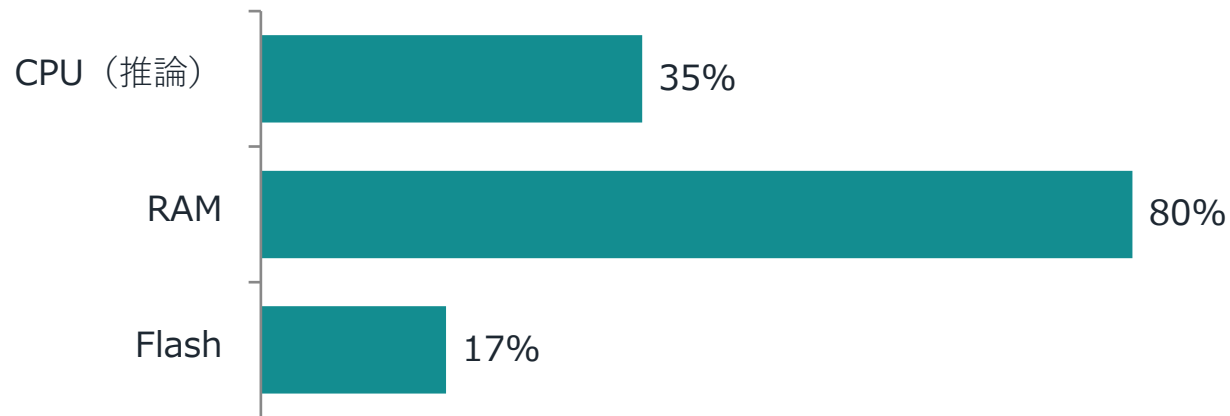
85KB

Flash使用量 (512KB中)

7,043

モデルのパラメータ数

容量に対する使用率



通信なし・電池で動作

- 音声は端末の外に一切出ない
- クラウドもスマートフォンも使わない
- 単4電池2本で動く

誤報を防ぎつつ、短い音にも反応する

通知する条件（音の種類ごとに変える）

音	通知する条件	理由
サイレン	確信度60%以上が 2回続いたら	鳴り続ける音なので、 慎重に判定しても遅れない
クラクション	確信度70%以上なら1回で (60%以上なら2回続いたら)	1秒以内に鳴り終わる 短い音なので

- 話し声・拍手・物音を5分間聞かせても、誤報は0回
- 通知したら、LEDと振動を3秒間続ける

振動のパターンで音の種類を区別

サイレン：長く2回（0.5秒×2）



クラクション：短く3回（0.2秒×3）



サイレンが鳴り続けている間は、パターンを最後まで再生してから繰り返すので、振動だけで区別しやすい

わかっている弱点と、これから

弱点

短いクラクションを取りこぼす

0.7秒の音は10回中10回、0.3秒の音は10回中2回。
1秒の窓の境目にまたがると、音が薄まってしまう

小さな音には反応しにくい

周りの雑音に埋もれるほど小さな音（遠くのサイレンなど）は「その他」と判定されてしまう

省電力はまだ評価できていない

電池の持ち時間は未測定。録音タスクは2msごとに完了を確かめており、CPUを休ませる余地がある

これから

1. 判定の窓を0.5秒ずつ重ねる

どこで鳴っても、どれかの窓に音がまとまって入る。
推論が約0.35秒なので、追加のメモリなしで可能

2. SAADCの完了を割り込みで待つ

待っている間はCPUを休ませ、電池を長持ちさせる

3. 知らせる音の種類を増やす

火災警報・踏切・ドアチャイムなど、
生活の中の大切な音へ用途を広げる

4. 屋外で録った実物の音でも学習

実物のクラクションや遠くのサイレンなど、
小さな音・低い音も学ばせる

まとめ

1. μ T-Kernelのタスク分割で、AIの推論中でも録音を止めないリアルタイム処理を実現した
2. RMS正規化と実機録音により、実機データで誤報ゼロ・サイレンの的中率100%を達成した
3. micro:bit単体・電池駆動・通信なしで動く、エッジAIウェアラブルを作った

ソースコード : <https://github.com/kazu0503/tron-sound-assist> (MITライセンス)

デモ動画 : <https://youtu.be/pyXOdZmFe1g>

渡邊 和弥 | 京都工芸繊維大学 工芸科学部 電子システム工学課程 3年